

A Theory of Programming

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Introduction

This lecture proposes a stepwise introduction to a foundation of programming and the analyses to write reliable and efficient software.

There are many different frameworks. We choose here a formal logic where relations and functions are the main building blocks. This entails that the meaning of **functional programs** can often be made very precise. (We will cover imperative programs later.)

It is a good framework to get a sense of **static analyses**, and, as such, it stands a prominent place in the field of formal methods and **compiler construction**.

A mini functional language

Rewrite systems are a good mathematical and mental model for understanding functional languages.

It is time now to learn an actual functional language, so we can run programs. We are going to start with a mini language that is actually a subset of OCaml, then grow it step by step.

Within rewrite systems, a 'program' is simply a term that is reduced until reaching a stuck term. While this is a good theoretic framework to think about computation in general, it is not conducive to programming in the large, nor understanding the behaviour of programs.

We want programs to be made of different parts that can be reused in different contexts, for example, variables should be defined somewhere and reused at some another points.

Growing a language

This presentation is principled, that is, it relies on a systematic classification of the **syntactic constructs** and their **formal semantics**.

When we reach a certain complexity, we will drop the formal descriptions.

Both theoretical and pragmatic, our approach is complementary to those usually found in books for general audiences.

Let mini-ML be the name of the subset of OCaml we are going to grow.

Appending a stack

The notation we introduced for stacks in the context of rewrite systems actually follows the OCaml convention.

In the context of functional programming, stacks are often called **lists**.

Here are different ways to express the same list in OCaml:

$$1 :: (2 :: 3 :: (4 :: [])) = 1 :: 2 :: 3 :: 4 :: [] = [1; 2; 3; 4].$$

Before starting, let us see how a familiar function defined with a rewrite system looks like in OCaml.

Let us recall the appending of a stack:

$$\begin{aligned}\text{cat}([], t) &\xrightarrow{\alpha} t; \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t).\end{aligned}$$

Appending a stack

In OCaml, the function name is not repeated (once for each rule).

Instead, it is introduced once by a **binder** called '**let**', like so:

let cat = **function** ...

The complete definition in OCaml is:

let rec cat = **function**

[], $t \rightarrow t$

| $x :: s, t \rightarrow x :: \text{cat}(s, t)$

Note the addition of the keyword **rec**: this is meant so the call $\text{cat}(s, t)$ indeed refers to the function being defined.

With rewrite systems, recursion is not an issue: any call matching a pattern can be rewritten.

Note also how the two rules are introduced by the keyword **function** and are separated by a vertical line.

Sentences

An OCaml program is a sequence of sentences. A **sentence**, or **global definition**, is defined by the following cases, where e denotes an expression, x is a variable, and '**let**' and '**rec**' are keywords:

- *global definition* **let** $x = e$
- *global recursive definition* **let rec** $x = e$

Global definitions can be optionally followed by two semicolons: these are necessary when inputting the sentences in the toplevel loop (prompted after running the command `ocaml` from a Unix shell).

Note that the keyword '**rec**' is necessary to enable recursion. (We will see later how that works precisely.)

For e to be an expression means that e denotes a part of a sentence which is classified as an expression according to its **syntax**.

Metavariables

- We say that e is a **metavariable** because, being a name, it is a variable, but that variable does not belong to the language being described (the **object language**, here, OCaml) and, instead, exists only in the descriptive language (the **metalanguage**).
- In the following

let $x = e$

the metavariable x (mind the italics) denotes an infinite set of OCaml variables and we must not confuse it with the OCaml variable x (no italics). Similarly, the metavariable e denotes an infinity of expressions, as in the following:

let $e = 3$

let $x = e$

let $a = x + e$

Expressions

- An expression e is **recursively defined by** the following **cases**:
 - *variable* $\text{my_var, } x, y, z, \text{ etc.}$
 - *unit or integer constant* $() \text{ or } 0 \text{ or } 1 \text{ or } 2, \text{ etc.}$
 - *function (or abstraction)* **fun** $x \rightarrow e$
 - *call (or application)* $e_1 \ e_2$
 - *arithmetic operator* $(+) \ (-) \ (/) \ (*)$
 - *arithmetic operation* $e_1 + e_2 \text{ or } e_1 - e_2, \text{ etc.}$
 - *parenthesis* (e)
 - *local definition* **let** $x = e_1 \text{ in } e_2$
- Note how the keywords '**rec**' and '**function**', as well as the definition of multiple cases (as seen in the definition of 'cat' earlier) is not found above: this is because we want to focus on other aspects of OCaml first, like higher-order functions.
- Instead, we have '**fun**', which enables only one case made of a variable (more generally, irrefutable patterns).

Examples of expressions

- Note that what we print nicely as ' $\rightarrow$ ' is actually written ' $->$ ' in the source code.
- Here are examples of OCaml sentences making up a program, step by step.
- First, a numerical constant:

let $x = 0$

- Note that constants can be defined by means of a rewrite system as functions taking the empty tuple:

$x() \rightarrow 0$

- The next sentence is

let $id = \text{fun } x \rightarrow x$

- This is equivalent to the rewrite system

$id(x) \rightarrow x$

Examples of expressions

- Next we have another constant being defined by binding a variable to the value of a call:

let $y = 2$ **in** $\text{id } y$

This is equivalent to the call $\text{id}(2)$.

- Now

let $x = (\text{fun } x \rightarrow \text{fun } y \rightarrow x + y)$ $1\ 2$

is equivalent to

let $x = 1 + 2$

(We will see why later.)

- Finally

let $z = x + 1$

is obvious.

Functions

- Variables must start with a lowercase character.
- The arrow is right-associative, so the expression

$$\mathbf{fun} \ x_1 \rightarrow \mathbf{fun} \ x_2 \rightarrow \dots \rightarrow \mathbf{fun} \ x_n \rightarrow e$$

is equivalent to

$$\mathbf{fun} \ x_1 \rightarrow (\mathbf{fun} \ x_2 \rightarrow (\dots \rightarrow (\mathbf{fun} \ x_n \rightarrow e)) \dots),$$

or, even shorter:

$$\mathbf{fun} \ x_1 \ x_2 \dots x_n \rightarrow e.$$

- Function calls are left-associative, so

$$e_1 \ e_2 \ e_3 \dots e_n \triangleq (((\dots (e_1 \ e_2) \ e_3) \dots) \ e_n).$$

- Function calls have higher priority than operator calls, for example,

$$f \ 3 + 4 = (f \ 3) + 4 = f(3) + 4.$$

Function calls

Operator calls have higher syntactic priority than abstractions, for instance,

fun $x \rightarrow x + y$ is the same as **fun** $x \rightarrow (x + y)$.

Note the difference between $f \times y$ and $f(x, y)$.

The former is to be understood as $(f(x))(y)$ which means that there are *two* calls:

1. First, the call $f(x)$ is evaluated. Let us assume that its value is g .
2. Second, the call $g(y)$ is evaluated.

In other words, the first case is equivalent to

let $g = f \times$ **in** $g \ y$

Abstract syntax

A graphical representation of programs (here, expressions) as trees proves very handy when studying their semantics (meaning):

Flat view	Tree view
x	x
fun $x \rightarrow e$	fun $\swarrow \searrow$ $x \quad e$
$e_1 \ e_2$	$\$$ $\swarrow \searrow$ $e_1 \ e_2$

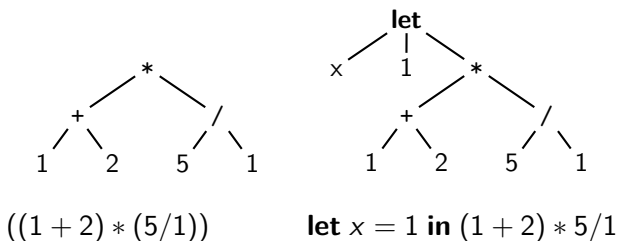
Note that we needed some symbol for the root of the application tree ($\$$). Traditionally, that symbol is ($@$), but OCaml uses it already for something else.

When trees represent programs, they are called **abstract syntax trees** (AST).

Abstract syntax

Intuitively, the method for constructing abstract syntax trees consists in fully parenthesising the expression. Each parenthesis corresponds to a subexpression and each subexpression corresponds to a subtree.

The tree is built from the root, down to the leaves, by traversing subexpressions from the outermost to the innermost, that is, the most embedded ones. For instance, consider

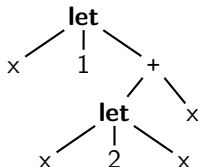


Abstract syntax

Another example:

let $x = 1$ **in** ((**let** $x = 2$ **in** x) + x)

corresponds to the abstract syntax tree



This example features two local definitions of 'x': is one the redefinition of the other? Is one hiding the other?

There are also two uses of 'x': which use corresponds to which definition?

Bindings, environments, scope

To answer these questions, we must first understand

- what a **binding** is,
- what the **scope** of a binding is, and
- what an **environment** is.

The expression we called **local definition** earlier:

$$\text{let } x = e_1 \text{ in } e_2$$

is evaluated as follows:

1. The expression e_1 is evaluated,
2. assuming it has a value v_1 , then v_1 is **bound** to the variable x ,
3. the expression e_2 is evaluated under the assumption that x is bound to v_1 .

Bindings, environments, scope

A local definition

let $x = e_1$ **in** e_2

associates the value v_1 of expression e_1 to the variable denoted by x , making up a **binding** noted $x \mapsto v_1$, and then proceeds to evaluate e_2 with this binding available.

The binding $x \mapsto v_1$ is not usable outside the evaluation of e_2 . We say that the **scope** of the definition is made of the subtrees of the abstract syntax tree corresponding to e_2 where the binding of that definition is usable ('visible').

A mapping from variables to values is called an **environment**. Let us note $\rho_\emptyset$ the empty environment. We write

$$\rho[x \mapsto v]$$

to mean that the binding $x \mapsto v$ was added to the environment ρ .

Bindings, environments, scope

Environments are functions and are updated as follows:

$$\rho[x \mapsto v](y) \triangleq \begin{cases} v & \text{if } x = y, \\ \rho(y) & \text{otherwise.} \end{cases} \quad (1)$$

with $\forall x. \rho_{\emptyset}(x) = \perp$ and $\perp$ is interpreted as an **error**. Errors are special values. Consider again the program

```
let x = 0 in  
let id = fun x  $\rightarrow$  x in  
let y = id x in  
let x = (fun x y  $\rightarrow$  x + y) 1 2  
in x + 1
```

The variable x in the expression $x+1$ denotes the value of the expression bound to the variable x in the previous line, not the first.

Bindings, environments, scope

Bindings are ordered in the environment **by the order of their definitions**. Let us follow the evolution of the environment.

1. The environment is initially empty: $\rho_{\emptyset}$;
2. **let** $x = 0$ **in** ...
yields $\rho_{\emptyset}[x \mapsto 0]$
3. **let** $id = \text{fun } x \rightarrow x$ **in** ...
yields $\rho_{\emptyset}[x \mapsto 0][id \mapsto \text{fun } x \rightarrow x]$;
4. **let** $y = id\ x$ **in** ...
yields $\rho_{\emptyset}[x \mapsto 0][id \mapsto \text{fun } x \rightarrow x][y \mapsto 0]$;
5. **let** $x = \dots$
yields $\rho_{\emptyset}[x \mapsto 0][id \mapsto \text{fun } x \rightarrow x][y \mapsto 0][x \mapsto 3]$.
6. The expression $x+1$ is evaluated in $3 + 1 = 4$ because $x \mapsto 3$ (the last binding) **hides** $x \mapsto 0$ (the first binding).
7. In other words, the first definition of x was **out of scope** in $x+1$.

Free variables

- If a variable x is not bound to a value in an environment ρ , we say that x is **free** in ρ .
- By definition, a variable is **free** in an expression if it is not bound by a **'let'** or a **'fun'**.
- We can understand this concept on a graphic representation of the abstract syntax tree.
- From each variable occurrence, we move up, towards the root.
- If we encounter a **'let'** that binds that variable (in the left child), we create an oriented edge from the variable to that **'let'**, and we stop: the variable is bound.
- Otherwise, if we reach the root (no such **'let'** has been found), then the variable is free.

Free variables

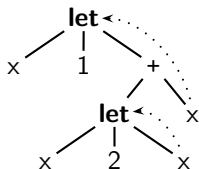


Figure: `let x = 1 in ((let x = 2 in x) + x)`

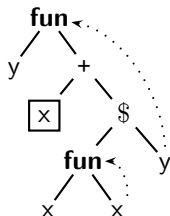


Figure: `fun y -> x + (fun x -> x) y`

Free variables

A similar situation arises with functions: '**fun**' is a binder, just like '**let**'.

In **fun** $x \rightarrow e$, the variable x (called a **parameter**) may shadow (hide) another variable x defined on the path to the root.

See previous page for two examples.

We note $\mathcal{F}(e)$ the set of the variables free in e .

Free variables

It is easy to formally define $\mathcal{F}$ by case and recursively:

$$\mathcal{F}(x) = \{x\}, \quad \text{where } x \text{ is a variable}$$

$$\mathcal{F}(c) = \emptyset, \quad \text{where } c \text{ is a constant}$$

$$\mathcal{F}(\text{fun } x \rightarrow e) = \mathcal{F}(e) \setminus \{x\}$$

$$\mathcal{F}(e_1 \ e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2)$$

$$\mathcal{F}(\text{let } x = e_1 \text{ in } e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2) \setminus \{x\}$$

$$\mathcal{F}(e_1 + e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2)$$

Now, instead of drawing abstract syntax trees, let us use these equations to compute the free variables of our previous examples.

Free variables

$$\begin{aligned}\mathcal{F}(\text{let } x = 1 \text{ in } (\text{let } x = 2 \text{ in } x) + x) \\&= \mathcal{F}(1) \cup \mathcal{F}((\text{let } x = 2 \text{ in } x) + x) \setminus \{x\} \\&= \emptyset \cup (\mathcal{F}(\text{let } x = 2 \text{ in } x) \cup \mathcal{F}(x)) \setminus \{x\} \\&= (\mathcal{F}(2) \cup \mathcal{F}(x) \setminus \{x\} \cup \mathcal{F}(x)) \setminus \{x\} \\&= \mathcal{F}(x) \setminus \{x\} \\&= \emptyset.\end{aligned}$$

$$\begin{aligned}\mathcal{F}(\text{fun } y \rightarrow x + (\text{fun } x \rightarrow x) y) \\&= \mathcal{F}(x + (\text{fun } x \rightarrow x) y) \setminus \{y\} \\&= (\mathcal{F}(x) \cup \mathcal{F}((\text{fun } x \rightarrow x) y)) \setminus \{y\} \\&= (\{x\} \cup \mathcal{F}(\text{fun } x \rightarrow x) \cup \mathcal{F}(y)) \setminus \{y\} \\&= (\{x\} \cup \mathcal{F}(x) \setminus \{x\} \cup \{y\}) \setminus \{y\} \\&= \{x, y\} \setminus \{y\} \\&= \{x\}.\end{aligned}$$

Closed expressions

- A **closed** expression is an expression without free variables.
- That is, an expression e is closed if, and only if, $\mathcal{F}(e) = \emptyset$.
- By extension, we say that e is **closed in an environment** ρ if $\mathcal{F}(e) \subseteq \text{dom}(\rho)$ (the free variables are bound in ρ).
- Only a closed expression can be evaluated (executed). That is why the first static analysis performed by compilers consists in determining the variables which are free in expressions: if the program is not closed, it is rejected.
- In the case of

fun $y \rightarrow x + (\text{fun } x \rightarrow x) y$,

the OCaml compiler prints out

Unbound value x

and stops. It is useful that this open expression be rejected at compile-time and does not cause a run-time error.

Evaluation

Evaluation is a partial function from expressions to values. An **interpreter** is a program implementing an evaluation. The partiality models the fact that an evaluation may not terminate or end in an error.

With rewrite systems, values are defined as stuck expressions, that is, expressions that cannot be rewritten further.

Here, since we want to define expressions (terms) whose value we can reuse somewhere else (in another term), we need the notion of **environment**.

That implies that a formal account of evaluation (the equivalent for programs of rewriting terms) needs to take into account environments.

Semantic values

This goes even further because we want Mini-ML to be **higher-order**, that is, we want functions to be values and this begets the question of the meaning of the free variables in the function bodies: after passing around a function as argument to other functions, what are the values of those variables when the threaded function is finally applied?

The most common answer is **lexical (or static) scoping**, whereby the free variables in a body remain bound to their values at the point the function was defined.

This means that values corresponding to functions have to carry around the environment at the point of their definition.

One common way to achieve this is to have a notion of **semantic value**, which differs from **syntactic value**. For example, the mathematical integers are part of the semantic values of Mini-ML, but not of syntactical values.

Semantic values

- The **semantic values** are defined recursively by the following cases:
 - *constant* $()$, 0, 1, 2, etc.
 - *closure* $\langle \mathbf{fun} \ x \rightarrow e; \rho \rangle$, with environment ρ .
 - *n-tuple* $v_1, \dots, v_n$, with v_i values.
- The closure environment ρ is the one at the location where the function **fun** $x \rightarrow e$ occurs, and the function is closed in that environment (this is called **static scoping**, or **lexical scoping**).
- The evaluation of an expression must take place in the context of an environment, where all free variables must be bound (in other words, the expression must be closed in that environment).
- For example, let ρ be the environment before the sentence

let $x = 1$ **in** x

Then the evaluation of the *second* ' x ' must be done in the environment $\rho[x \mapsto 1]$, where ' x ' is the *first* one.

Evaluation

Let us proceed to define a semantics for our mini-ML, by associating to each expression e its value v , assuming the environment ρ :

- x Look up the **last** value bound to x in ρ .
- **fun** $x \rightarrow e$ The value is the closure $\langle \text{fun } x \rightarrow e; \rho \rangle$.
- $e_1 + e_2$, etc. Evaluate e_1 and e_2 in ρ , then add, etc.
- $()$ or 0 or 1 or 2, etc. The value is $()$ or 0 or 1, etc.
- (e) Evaluate e into v in ρ .
- **let** $x = e_1$ **in** e_2 Evaluate e_1 into v_1 in ρ ;
evaluate e_2 into v in $\rho[x \mapsto v_1]$.
- $e_1 \ e_2$ Evaluate e_1 and e_2 into v_1 and v_2 in ρ
(order left unspecified);
 v_1 must be the closure $\langle \text{fun } x \rightarrow e; \rho_1 \rangle$;
evaluate e in $\rho_1[x \mapsto v_2]$;
the value is v .

Example of evaluation

1. The environment is initially empty. Let us evaluate

```
let x = 0 in  
let id = fun x  $\rightarrow$  x in  
let y = id x in  
let x = (fun x y  $\rightarrow$  x + y) 1 2  
in x + 1
```

2. The evaluation of

```
let x = 0 in ...
```

starts with the evaluation of 0, which, obviously, yields 0.

3. We create the binding $x \mapsto 0$, with which we extend $\rho_\emptyset$, that is, $\rho_\emptyset[x \mapsto 0]$.
4. We evaluate the elided subexpression within this new environment.

Example of evaluation

- The evaluation of

let id = **fun** x $\rightarrow$ x **in** ...

is performed within the environment $\rho_{\emptyset}[x \mapsto 0]$.

- The value of 'id' is then the closure $\langle \text{fun } x \rightarrow x; \rho_{\emptyset}[x \mapsto 0] \rangle$.
- We extend the current environment and evaluate the elided subexpression within it.
- The evaluation of

let y = id x **in** ...

is done within the environment

$\rho = \rho_{\emptyset}[x \mapsto 0][\text{id} \mapsto \langle \text{fun } x \rightarrow x; \rho_{\emptyset}[x \mapsto 0] \rangle]$.

- We evaluate first (id x) within ρ .
- In order to do so, we evaluate 'id' and 'x' separately.

Example of evaluation

- These are both variables, thus we look them up in the environment to retrieve their last corresponding binding:
 $\rho(\text{id}) = \langle \text{fun } x \rightarrow x; \rho_{\emptyset}[x \mapsto 0] \rangle$ and $\rho(x) = 0$.
- We need to evaluate the body in the environment $\rho' = \rho_{\emptyset}[x \mapsto 0][x \mapsto 0]$, which yields $\rho'(x) = 0$.
- We create the binding $y \mapsto 0$, we add it to the current environment and we evaluate the elided subexpression with the new environment.
- Let us stop here and move to expressing the above semantics in a formal manner.

Formal semantics

If we note $\llbracket e \rrbracket \rho$ the value obtained by evaluating the expression e in the environment ρ , then the evaluation of mini-ML is:

$$\llbracket \bar{n} \rrbracket \rho = n, \text{ with } \bar{n} \text{ an OCaml integer and } n \in \mathbb{N}$$

$$\llbracket e_1 + e_2 \rrbracket \rho = \llbracket e_1 \rrbracket \rho + \llbracket e_2 \rrbracket \rho \in \mathbb{N}$$

$$\llbracket (e) \rrbracket \rho = \llbracket e \rrbracket \rho$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \mathbf{fun} \ x \rightarrow e \rrbracket \rho = \langle \mathbf{fun} \ x \rightarrow e; \rho \rangle$$

$$\begin{aligned} \llbracket e_1 \ e_2 \rrbracket \rho &= \llbracket e_3 \rrbracket \rho_3 [x \mapsto \llbracket e_2 \rrbracket \rho], \\ &\text{where } \llbracket e_1 \rrbracket \rho = \langle \mathbf{fun} \ x \rightarrow e_3; \rho_3 \rangle \end{aligned}$$

$$\llbracket \mathbf{let} \ x = e_1 \ \mathbf{in} \ e_2 \rrbracket \rho = \llbracket (\mathbf{fun} \ x \rightarrow e_2) \ e_1 \rrbracket \rho.$$

Evaluation means applying the equations from left to right.

Partial applications

- We can easily work out that

$$\llbracket \text{let } x = e_1 \text{ in } e_2 \rrbracket \rho = \llbracket e_2 \rrbracket \rho[x \mapsto \llbracket e_1 \rrbracket \rho].$$

- Since closures are values, they can be the value of a function call:

```
let add = fun x y → x + y in
let incr = add 1
in incr 5
```

- The call (add 1) is a **partial application**, as opposed to a **complete application** like (add 1 5). A partial application, by definition, always evaluates in a closure.
- Operations are functions denoted by a symbol used in infix position (between the operands). As such, they can be partially evaluated:

```
let incr = (+) 1 in incr 5
```

where the parentheses in (+) mean that the operator has to be considered in **prefix** position.

Auto-application and termination

Our mini-ML is **Turing-complete**, just as OCaml is. For instance, we can write the following non-terminating program:

let $\Omega = \text{fun } x \rightarrow x \ x$ **in** $\Omega \ \Omega$

The function Ω is an **auto-applicative function**. It enables the following non-terminating evaluation:

$$\begin{aligned} \llbracket \text{let } \Omega = \text{fun } x \rightarrow x \ x \text{ in } \Omega \ \Omega \rrbracket \rho &= \llbracket \Omega \ \Omega \rrbracket \rho [\Omega \mapsto \llbracket \text{fun } x \rightarrow x \ x \rrbracket \rho] \\ &= \llbracket x \ x \rrbracket \rho [x \mapsto \langle \text{fun } x \rightarrow x \ x; \rho \rangle] \\ &= \textit{idem}. \end{aligned}$$

Fixed points

- To demonstrate the expressive power of our mini-ML, let us see how we can simulate recursive functions by means of the Ω function:

$$\text{let } \Omega = \text{fun } x \rightarrow x x$$

- Let us define a function `fix`, called the **Y combinator** in λ -calculus:

$$\text{let fix} = \text{fun } g \rightarrow \Omega (\text{fun } h x \rightarrow g (h h) x)$$

- For any (terminating) function f and any function x ,

$$\llbracket f (\text{fix } f) x \rrbracket \rho = \llbracket \text{fix } f x \rrbracket \rho.$$

- In other words, we have

$$f (\text{fix } f) x \equiv \text{fix } f x.$$

- The fixed point x of a (continuous) function g satisfies $g(x) = x$.
- Therefore, the fixed point of 'f' (if it exists) is the value of 'fix f'.

Fixed points

Just as with **let in**, we can simplify the semantics of ‘fix’ a bit:

$$\llbracket \text{fix } e \rrbracket \rho = \llbracket e_1 \rrbracket \rho_1 [x \mapsto \llbracket \text{fix } (\mathbf{fun } x \rightarrow e_1) \rrbracket \rho],$$

where $\llbracket e \rrbracket \rho = \langle \mathbf{fun } x \rightarrow e_1; \rho_1 \rangle$.

There is a choice:

- either we define ‘fix’ in the functional language (as a built-in constant), or
- at the meta-language (the mathematical logic we are using to define the semantics).

Fixed points

Let us consider the following definitions:

```
let pre_fact = fun f n  $\rightarrow$  if n = 1 then 1 else n * f(n - 1)  
let fact = fix pre_fact
```

We see that 'fact' is the fixed point of 'pre_fact':

$$\begin{aligned}\llbracket \text{fact} \rrbracket \rho &= \llbracket \text{fix pre_fact} \rrbracket \rho \\ &= \llbracket \text{pre_fact (fix pre_fact)} \rrbracket \rho \\ &= \llbracket \text{pre_fact fact} \rrbracket \rho \\ &= \llbracket \text{fun } n \rightarrow \text{if } n = 1 \text{ then } 1 \text{ else } n * \text{fact}(n - 1) \rrbracket \rho\end{aligned}$$

A non-recursive definition of the factorial.

Fixed points in C

```
#include<stdio.h>
#include<stdlib.h>

typedef int (*fp)();

int fact(fp f, int n) {
    return n? n * ((int (*)(fp,int))f)(f,n-1) : 1; }

int read(int dec, char arg[]) {
    return ('0' <= *arg && *arg <= '9')?
        read(10*dec + *arg - '0', arg+1) : dec; }

int main(int argc, char** argv) {
    if (argc == 2)
        printf("%u\n", fact(&fact, read(0, argv[1])));
    else printf("Only one integer allowed.\n");
    return 0; }
```

Local recursive definitions

- The function 'fact' coincides pointwise with the factorial function.
- Let us extend mini-ML with a native recursive binder, using the 'fix' function:

$$\llbracket \text{let rec } f = e_1 \text{ in } e_2 \rrbracket \rho = \llbracket \text{let } f = \text{fix } (\text{fun } f \rightarrow e_1) \text{ in } e_2 \rrbracket \rho.$$

Extensions

- Let us extend our mini-ML with the following expressions.
 - *Boolean constants* **true** or **false**
 - *Boolean operators* (**&&**) or (**||**) or **not**
 - *n-tuple* $e_1, \dots, e_n$
 - *conditional* **if** e_0 **then** e_1 **else** e_2
 - *local recursive binding* **let rec** $f = e_1$ **in** e_2
- Note that parentheses are recommended around tuples, but are not mandatory.
- Let us distinguish the variables occurring after **let** and **fun**, because they are **irrefutable patterns**, noted $\overline{p}$:
 - *function* **fun** $\overline{p} \rightarrow e$
 - *local definition* **let** [**rec**] $\overline{p} = e_1$ **in** e_2

Irrefutable patterns

- An irrefutable pattern $\bar{p}$ is defined recursively by the following cases:
 - *variable* f, g, h, x, y, z , etc.
 - *unit* $()$
 - *n-tuple* $\bar{p}_1, \dots, \bar{p}_n$
 - *parenthesis* $(\bar{p})$
 - *underscore* $_$
- Note that, from the syntactic point of view, irrefutable patterns are special cases of expressions, except the underscore, which is a special case avoiding the creation of a binding.
- The sentence e is equivalent to **let** $_ = e$.

Extensions

- The comma takes priority over the arrow:

fun $x \rightarrow x, y$ is the same as **fun** $x \rightarrow (x, y)$.

- In order to alleviate the notation **fun** $\bar{p}_1 \rightarrow \dots \rightarrow$ **fun** $\bar{p}_n \rightarrow e$, we define the equivalent expressions:
 - **let** [**rec**] $f =$ **fun** $\bar{p}_1 \bar{p}_2 \dots \bar{p}_n \rightarrow e$ (new expression)
 - **let** [**rec**] $f \bar{p}_1 \bar{p}_2 \dots \bar{p}_n = e$ (new sentence)
- For example, we would write

let **mult** = **fun** $x y \rightarrow x * y$ **in**

let $\text{eq } x y = (x = y)$ **in**

let rec **fact** $n =$ **if** $\text{eq } n 0$ **then** 1 **else** **mult** n (**fact** $(n - 1)$)

in **fact** 9

- Note the two different meanings of (=): one is definitional, the other is operational (a Boolean operator).

Parallel definitions

- We also extend the syntax to alleviate certain expressions. By definition,

let $\bar{p}_1 = e_1$ **and** $\bar{p}_2 = e_2 \dots$ **and** $\bar{p}_n = e_n$ **in** e

is equivalent to

let $\bar{p}_1, \dots, \bar{p}_n = e_1, \dots, e_n$ **in** e

but the latter requires more memory (for the tuples in the heap).

- These are called **parallel definitions** because they occur within the same environment and could be conceived as happening in parallel.
- The two following sentences are equivalent:

let $x = 1$ **and** $y = 2$ **in** $x + y$

let $x, y = 1, 2$ **in** $x + y$

Mutually recursive definitions

- Parallel definitions are useful for **mutually recursive definitions**:
 - **let rec** $\bar{p}_1 = e_1$ **and** $\bar{p}_2 = e_2 \dots$ **and** $\bar{p}_n = e_n$ **in** e (expression)
 - **let rec** $\bar{p}_1 = e_1$ **and** $\bar{p}_2 = e_2 \dots$ **and** $\bar{p}_n = e_n$ (sentence)
- A silly example:

let rec even $n =$ **if** $2 * (n/2) = n$ **then true else** odd($n - 1$)
and odd $n =$ **if** $2 * (n/2) = n$ **then false else** even($n - 1$)

where $(/)$ is the Euclidean division.